

Unsupervised Learning, Clustering, & Dimensionality Reduction

Nov. 18, 2025 - Lecture 9

Outline

- What is unsupervised learning?
- Clustering
- Clustering Algorithms
 - K-means, GMM, EM algorithm, Agglomerative Clustering, DBSCAN
- What is dimensionality reduction? Why do we need it?
 - PCA, UMAP, t-SNE, Autoencoders

Supervised vs. Unsupervised Learning

Supervised Learning = data + labels

Unsupervised Learning = data only, no labels (learn patterns from unlabelled data).

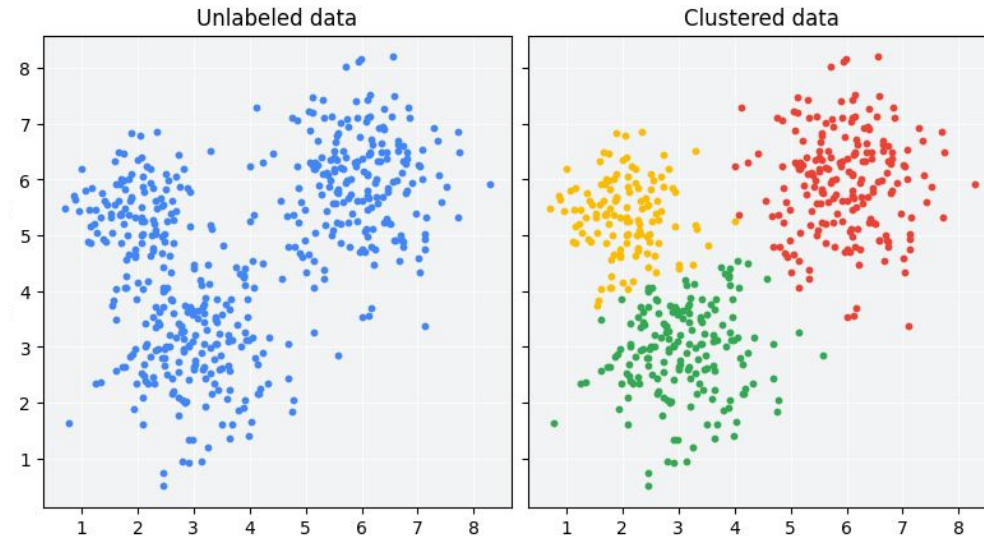
The objective of **unsupervised learning** is to **uncover hidden patterns, structures, or features** in the data.

In general, there are 3 types of unsupervised learning tasks: **clustering**, association rules, and **dimensionality reduction**.

Clustering

What is Clustering?

Clustering is a technique which **groups** unlabeled data (into clusters) based on their similarities or differences.

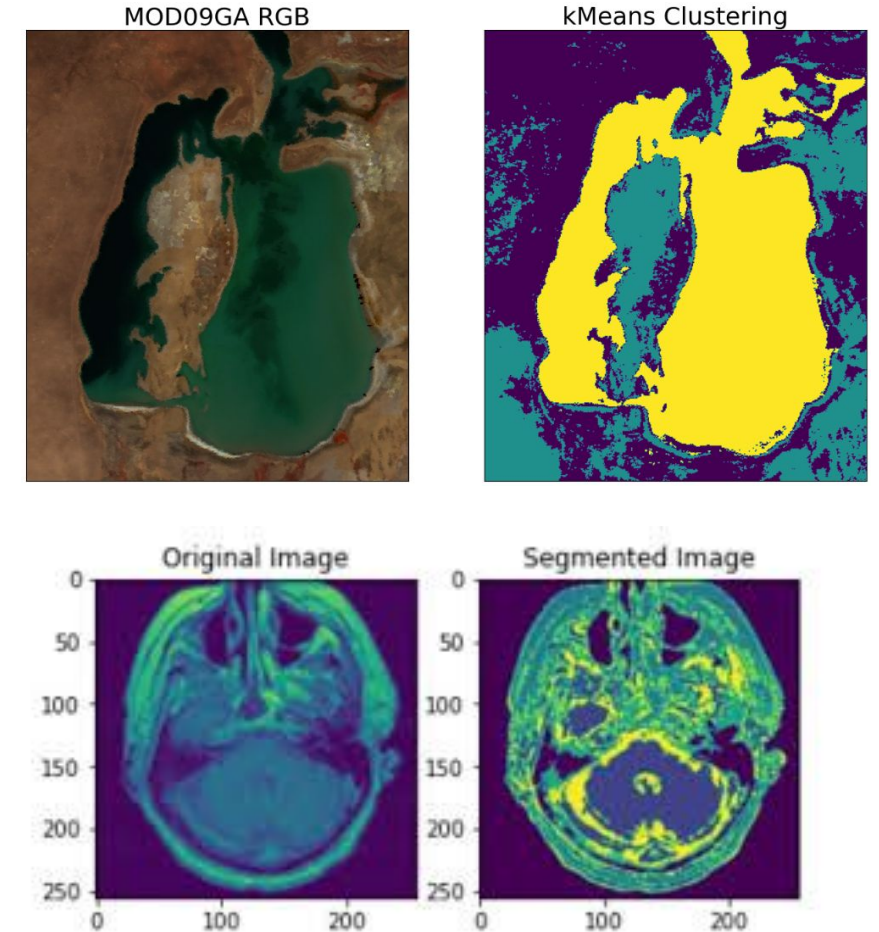


In the figure on the left, we can pretty easily identify that there are 3 clusters. However, in practice, we need to define a **similarity measure** that groups points together.

Clustering Examples

Clustering is useful in many industries.
Some applications of clustering are:

- Market segmentation
- Social network analysis
- Medical imaging
- Image segmentation
- Anomaly detection



Types of Clustering

The notion of a "cluster" is difficult to define. As a result, there are many approaches to clustering.

- **Exclusive Clustering:** a data point can only belong to a single cluster (e.g. k-means).
- **Overlapping Clustering:** a data point can exist in two or more clusters with different degrees of membership (e.g. fuzzy k-means).
- **Hierarchical Clustering:** data is repeatedly merged and organized based on their hierarchical relationships (e.g. agglomerative and divisive clustering).
- **Probabilistic Clustering:** data points are clustered based on the likelihood that they belong to a particular distribution (e.g. Gaussian mixture models).
- **Density-Based Clustering:** clusters are regions of high point density separated by regions of low density (e.g. DBSCAN, OPTICS).

Clustering Algorithms

Not all clustering algorithms are equal in **time complexity**. Many clustering algorithms compute the similarity between **all pairs of examples**, which means their runtime increases as the square of the number of examples ($O(n^2)$).

These algorithms are **not practical** for datasets with millions of examples. This is something to keep in mind when choosing a clustering algorithm for a specific dataset.

K-Means Clustering

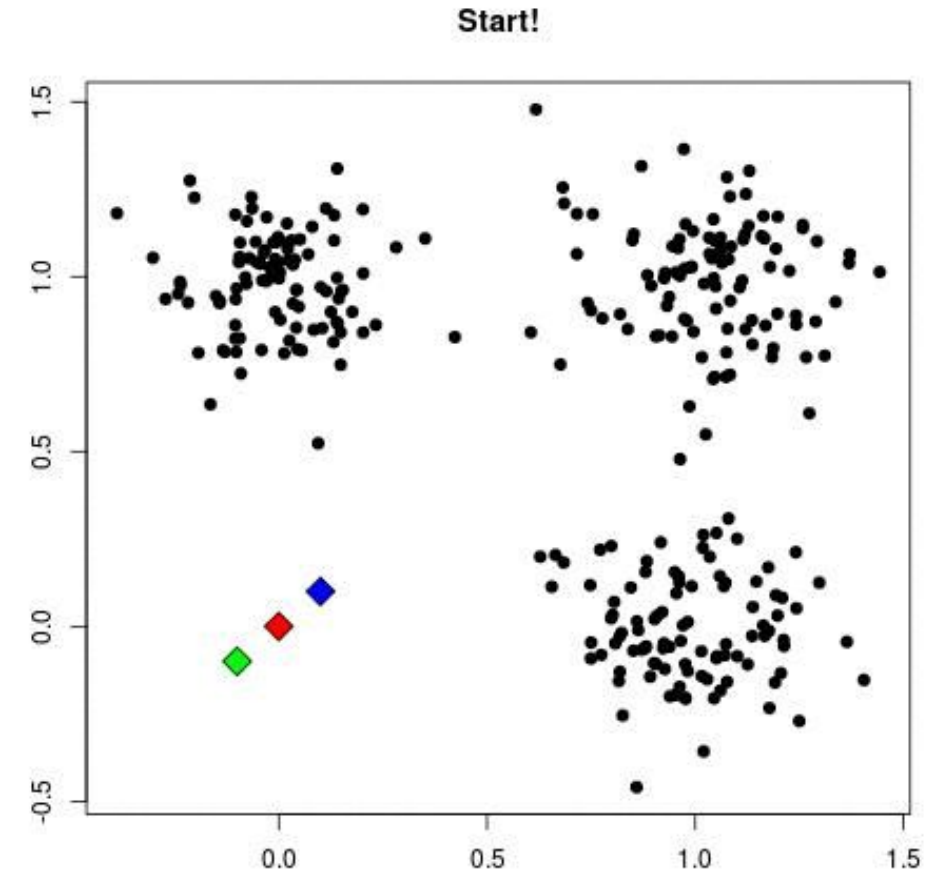
The K-means clustering algorithm assigns data points into k clusters by minimizing the distances between each point and its cluster's **centroid**. It has a time complexity of $O(nk)$.

- The **centroid** of a cluster is the arithmetic mean of all the points in the cluster.

K-means one of the **fastest** clustering algorithms, however, it can often fall into **local minima**. It is common to try several **random initializations** to see how it affects results.

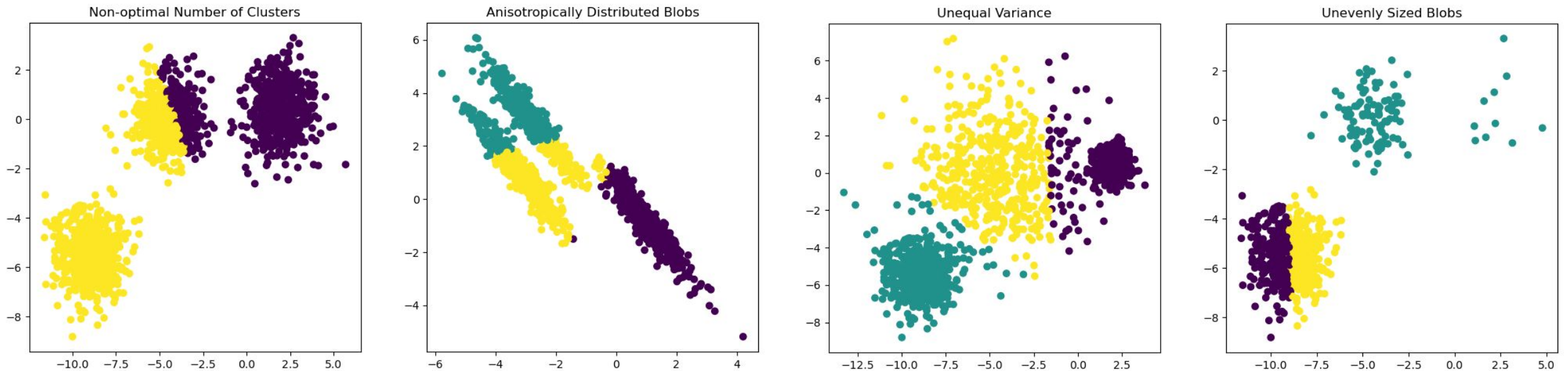
K-Means Clustering

1. Initialize k centroids randomly.
2. Assign each point to the nearest centroid to get initial clusters.
3. For each cluster, calculate a new centroid by taking the mean position of all points in the cluster.
4. Repeat 2-3 until the points stop changing clusters.



K-Means Clustering: Shortcomings

The plots below show some cases in which k-means **fails to distinguish between clusters**.



K-Means Clustering: Selecting k

There is no "golden rule" for selecting k , and there are several approaches to do it.

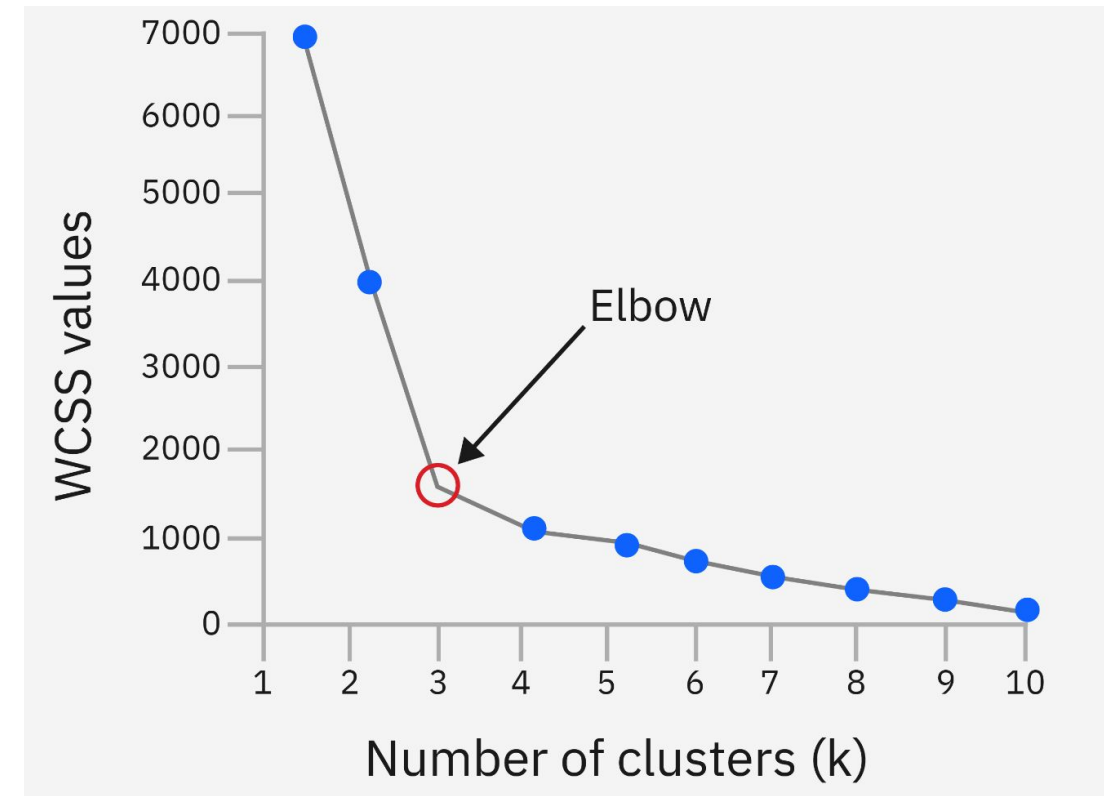
1. Use prior information / intuition (if applicable)
2. Elbow plot
3. Silhouette score / plots

Elbow Method

The **elbow method** is a **graphical method** for finding the optimum number of clusters for the k-means clustering algorithm.

We plot the sum of squared distances between each point and its cluster center (called within-cluster sum of squares, **WCSS**) against the number of clusters.

Using the plot, we pick the value of k where adding another cluster doesn't give much better results (the elbow).



Silhouette Scores / Plot

The **silhouette score** measures how well each point fits within its assigned cluster. For each point we compute how close it is to points in its own cluster (**cohesion**), and how far it is from points in the nearest other cluster (**separation**).

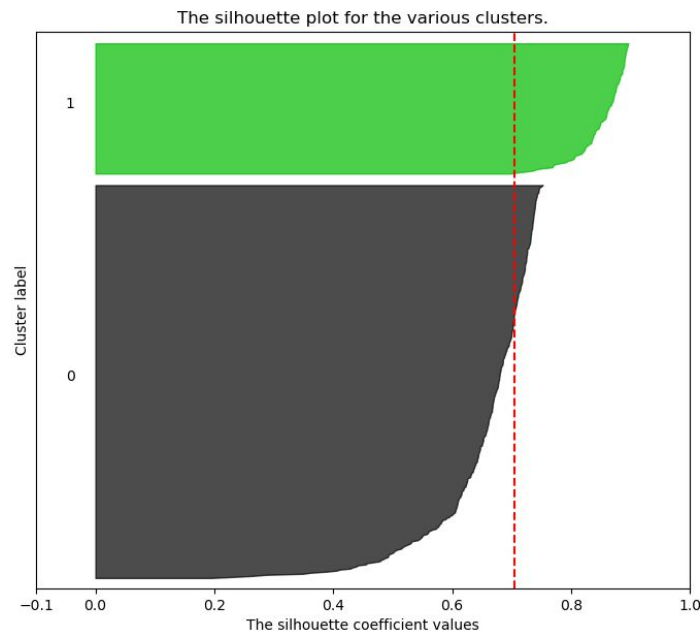
Silhouette scores range from $[-1, 1]$:

- Negative score $\rightarrow$ probably the wrong cluster
- Close to 0 $\rightarrow$ on the border between cluster
- Close to 1 $\rightarrow$ well-clustered

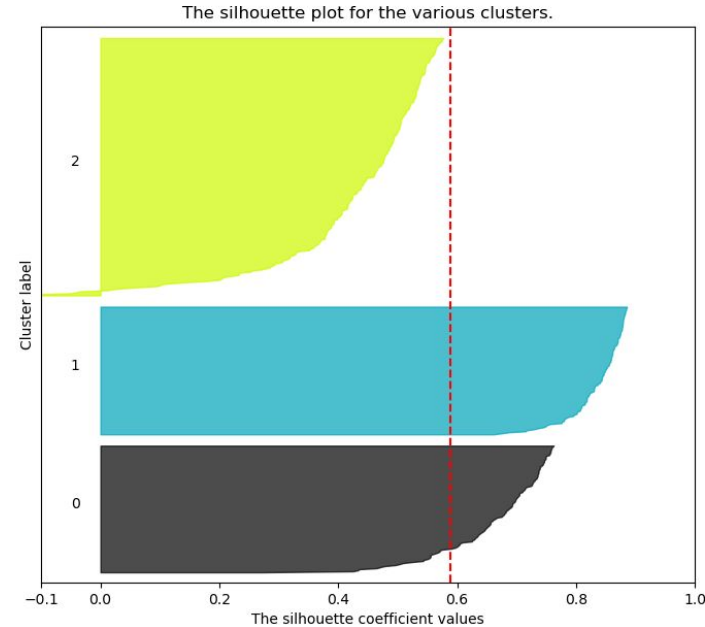
We **average the silhouette scores** across all points to show how good the clustering is. Typically, the k with the highest average silhouette score is usually best.

Silhouette Scores / Plot

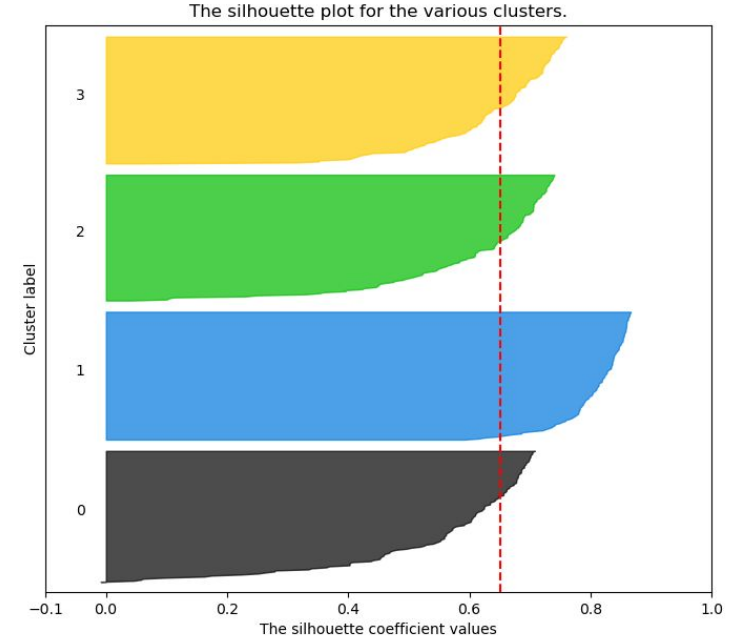
We can visualize this as well by plotting the silhouette scores. We want to see bars of equal width that approach a silhouette coefficient of 1. In the plots below, it looks like $k = 4$ matches this criteria, and therefore would be a good choice.



$k = 2$



$k = 3$



$k = 4$

K-Means Clustering

Pros:

- Fast and simple algorithm.

Cons:

- May not perform well for high-dimensional data - may want to use dimensionality reduction first.
- Bias toward **round, equally-sized** clusters.
- Cannot detect outliers.

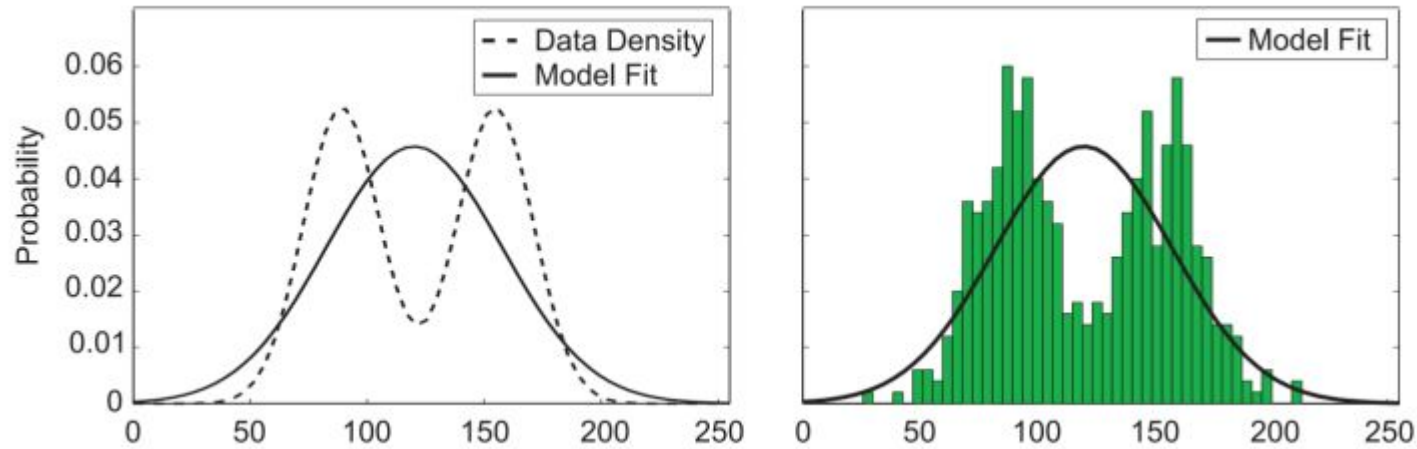
Gaussian Mixture Models (GMMs)

A **Gaussian mixture model (GMM)** is a **probabilistic model** that represents data as a combination (mixture) of several Gaussian distributions.

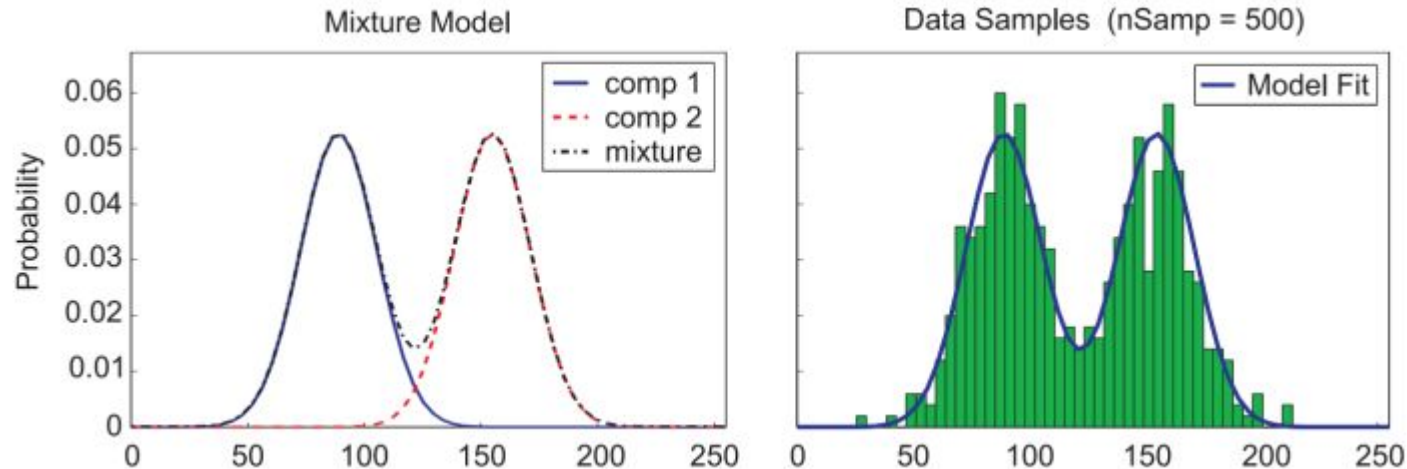
- A probabilistic view of clustering - each cluster corresponds to a different Gaussian component.
 - Example: Suppose you have data on people's heights with a bimodal distribution, but their gender identity is unknown. A GMM could model this as two Gaussian components (e.g. one for each group).
- Each data point has a mixing weight which determines the probability that it belongs to each Gaussian.
- GMMs are a general approach. The Gaussian can be replaced with other distributions to form mixture models (e.g. mixture of Poissons, Bernoullis, etc.).

Gaussian Mixture Models (GMMs)

One Gaussian is
not a good
approximation.



Two Gaussians
look much better
for this data.



Expectation Maximization (EM) Algorithm

The EM algorithm is a very common algorithm to optimize GMMs.

1. Initialize guesses for the model parameters (means, covariance matrices, and mixing weights).
2. **Expectation (E) Step:** Calculate the expected value of the log-likelihood function given the current parameter estimates (how much do we think each Gaussian generates each datapoint?).
3. **Maximization (M) Step:** Update the parameter estimates to maximize the expected log-likelihood in the E-step.
4. Repeat 2-3 until there is no change in the parameters or until a stopping criterion is reached.

GMMs

Pros:

- Can capture complex shapes.
- Can be easily extended to other distributions.

Cons:

- Rely on Gaussian (Normal) assumptions → do not perform well on non-elliptical clusters, or clusters with varying densities.
- Sensitive to initialization like k-means.
- Slower than k-means.

Agglomerative Clustering

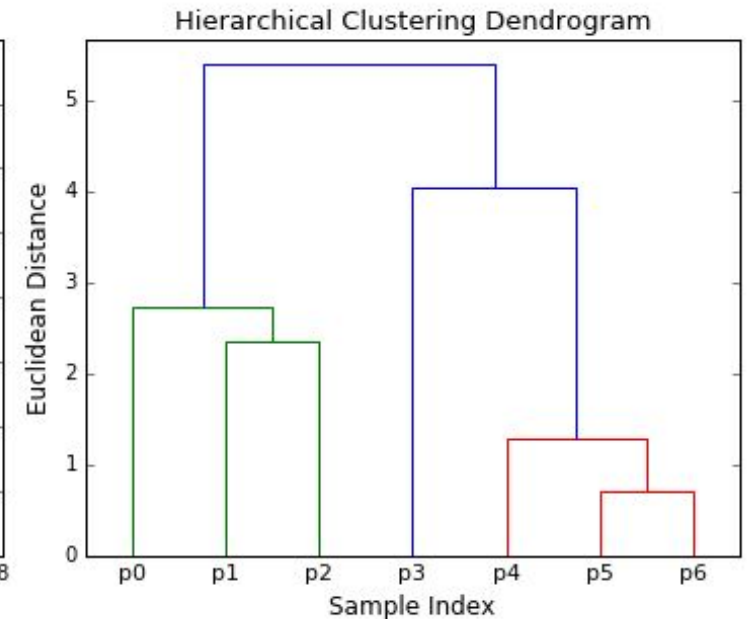
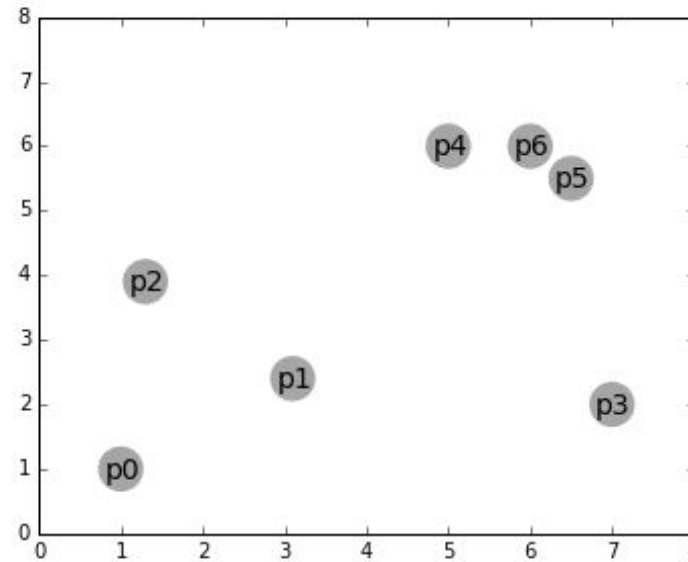
Agglomerative clustering is a method of hierarchical clustering using a bottom up approach: each observation starts in its own cluster, and clusters are successively merged together. There are different **merge strategies**:

- **Ward's linkage**: minimizes the sum of squared differences within all clusters.
- **Average linkage**: minimizes the average of the distances between all observations of pairs of clusters.
- **Complete / maximum linkage**: minimizes the maximum distance between observations of pairs of clusters.
- **Single / minimum linkage**: minimizes the distance between the closest observations of pairs of clusters.

The type of linkage used depends on your dataset and expected cluster shapes. Often average linkage is a good default.

Agglomerative Clustering

1. Treat each data point as its **own cluster**.
2. Calculate the pairwise distance (or similarity) between all clusters.
3. **Merge** the two clusters with the smallest distance according to the chosen linkage method.
4. **Update** distances between the new cluster and all remaining clusters.
5. Repeat steps 3-4 until only one cluster remains.



Agglomerative Clustering

Pros:

- No need to pre-specify number of clusters.
- Interpretable - dendrograms gives a clear visual of how clusters merge and relate hierarchically.

Cons:

- Sensitive to noise and outliers.
- Slow algorithm: $O(n^2)$ at best.

DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a clustering algorithm that groups together data points based on how densely packed they are.

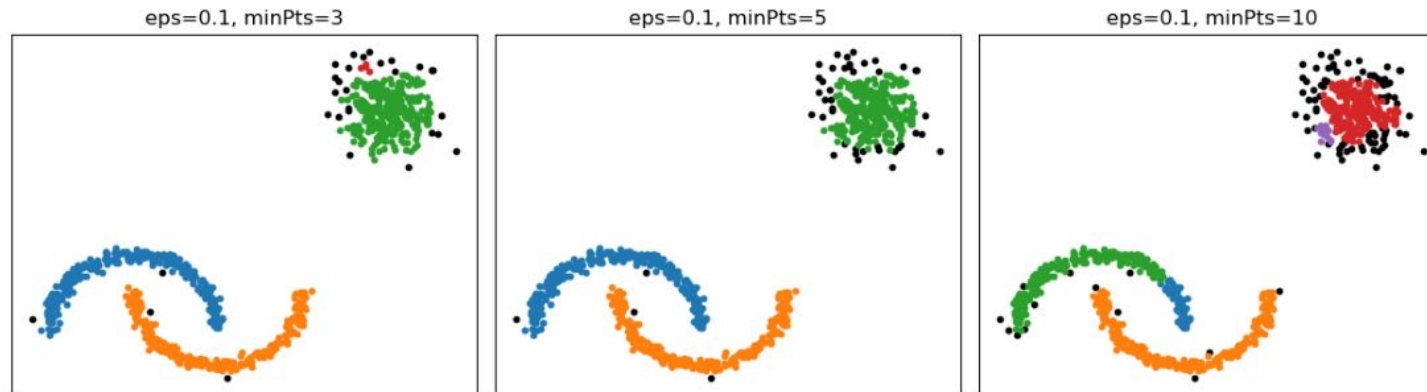
- Clusters form in **high-density areas** (points that are **closely packed** are marked as a cluster).
- Points that do not have enough neighbours are marked as **outliers**.
- Do not need to specify the number of clusters beforehand.
- DBSCAN can discover **clusters of arbitrary shape**.

DBSCAN

DBSCAN uses 2 parameters:

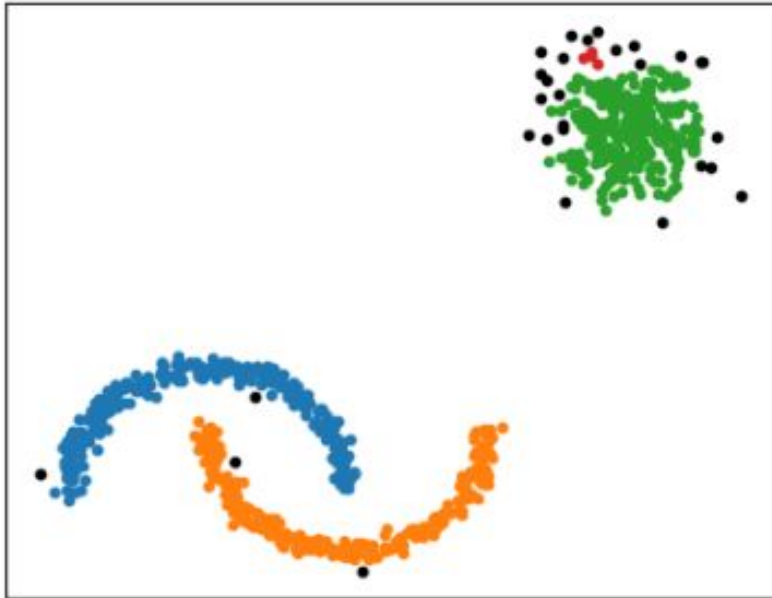
1. **ϵ (epsilon):** the max. distance between two points for them to be considered neighbors.
2. **minPts:** the minimum number of points needed to form a dense region.

These can be difficult to choose, and there is no optimal values for them. How to choose these values is beyond the scope of this lecture.

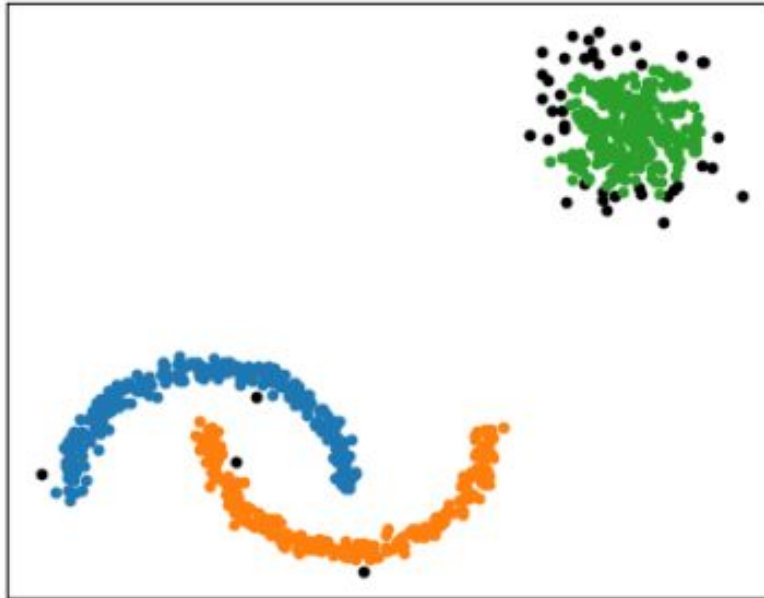


DBSCAN: Varying minPts

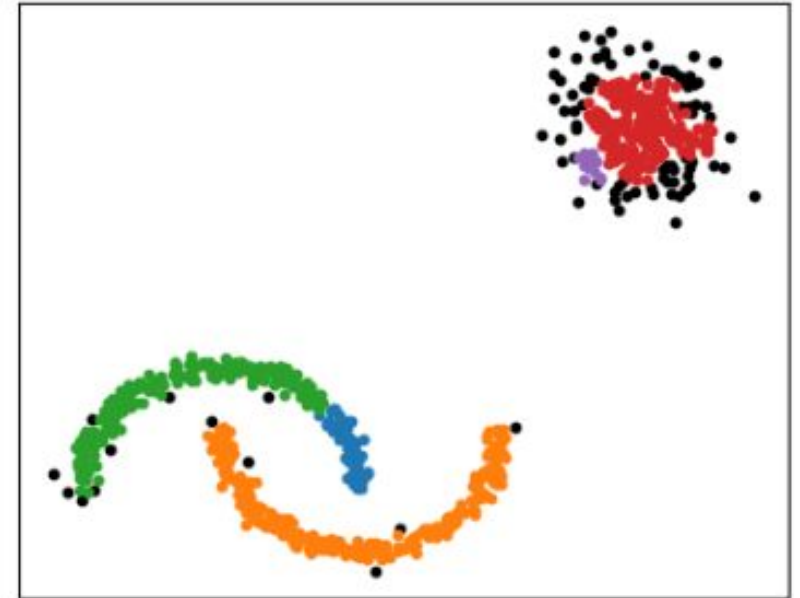
eps=0.1, minPts=3



eps=0.1, minPts=5

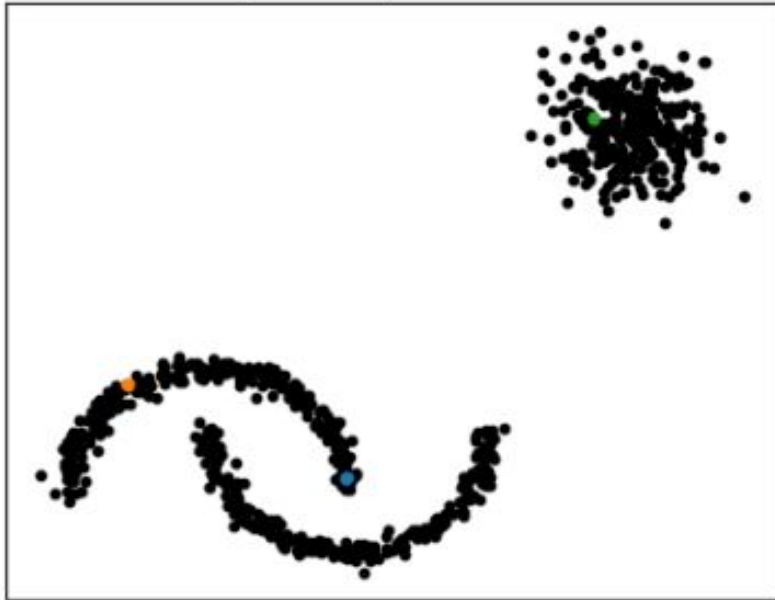


eps=0.1, minPts=10

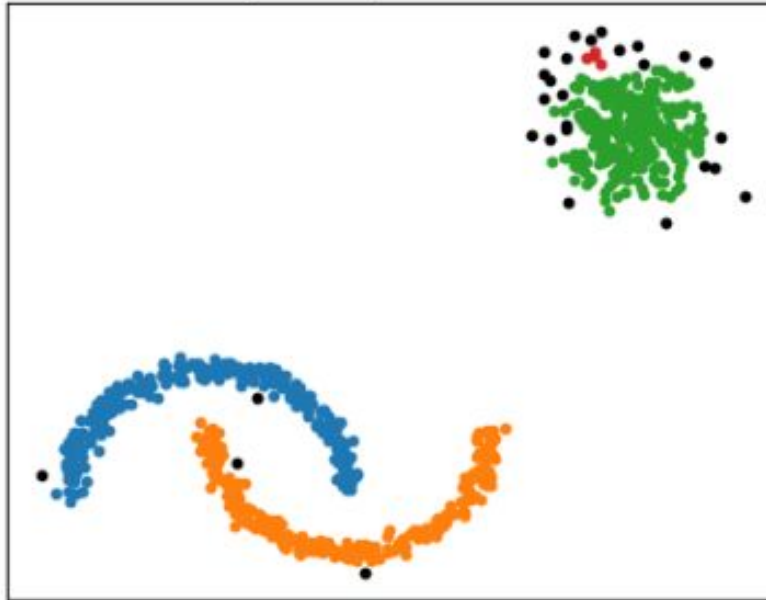


DBSCAN: Varying Epsilon

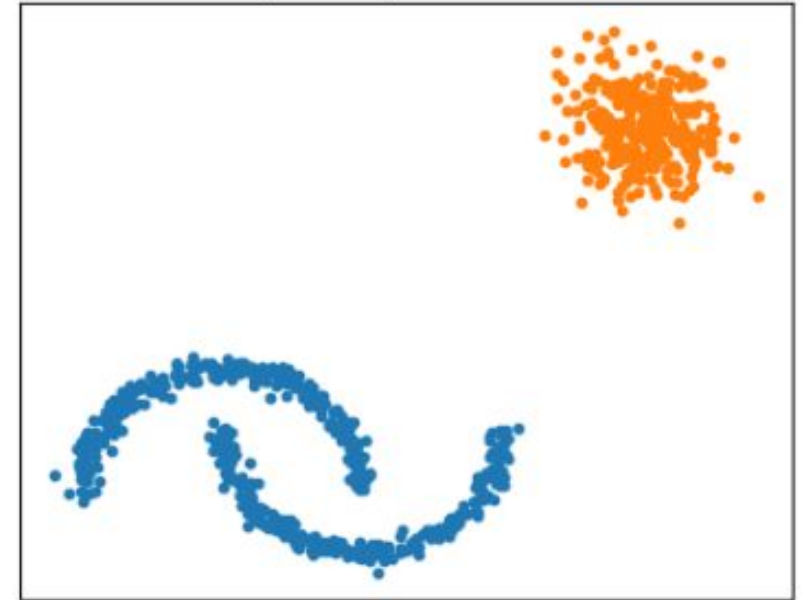
eps=0.01, minPts=3



eps=0.1, minPts=3



eps=0.5, minPts=3



HDBSCAN

Hierarchical DBSCAN (HDBSCAN) extends DBSCAN by converting it into a hierarchical clustering algorithm.

- It is better at **finding clusters with different densities** because it does not assume a single epsilon value.
- Its 2 main parameters are:
 1. **min_cluster_size**: size threshold for clusters.
 2. **min_samples**: strictness for labeling noise

DBSCAN and HDBSCAN

Pros:

- More robust to noise and outliers.
- Number of clusters don't need to be specified ahead of time.
- Can find arbitrarily-shaped clusters.

Cons:

- Assumes similar density of clusters (however HDBSCAN addresses this).
- Sensitive to parameter choices.
- Memory-intensive for large datasets (sklearn has a particularly slow implementation).

Evaluating Clusters

Supervised clustering:

- Can use accuracy, etc. since we have the actual cluster labels.

Unsupervised clustering is more tricky. Quality clusters contain at least 2 properties:

1. All data points within a cluster should be similar.
2. Clusters should be distinct from each other.

Evaluating Clusters: Unsupervised

Calinski-Harabasz Index: the ratio of the sum of between-clusters dispersion and of within-cluster dispersion for all clusters (where dispersion = sum of distances squared).

- Higher Calinski-Harabasz score = better defined clusters.

Davies-Bouldin Index: the average "similarity" between clusters, where the similarity is a measure that compares the distance between clusters with the size of the clusters themselves.

- Lower Davies-Bouldin index = better separation between clusters.

Evaluating Clusters

There are **several evaluation metrics for clustering**. Which to use depends on the clustering algorithm and type of problem (supervised vs. unsupervised).

For a more comprehensive list, see this [sklearn page](#).

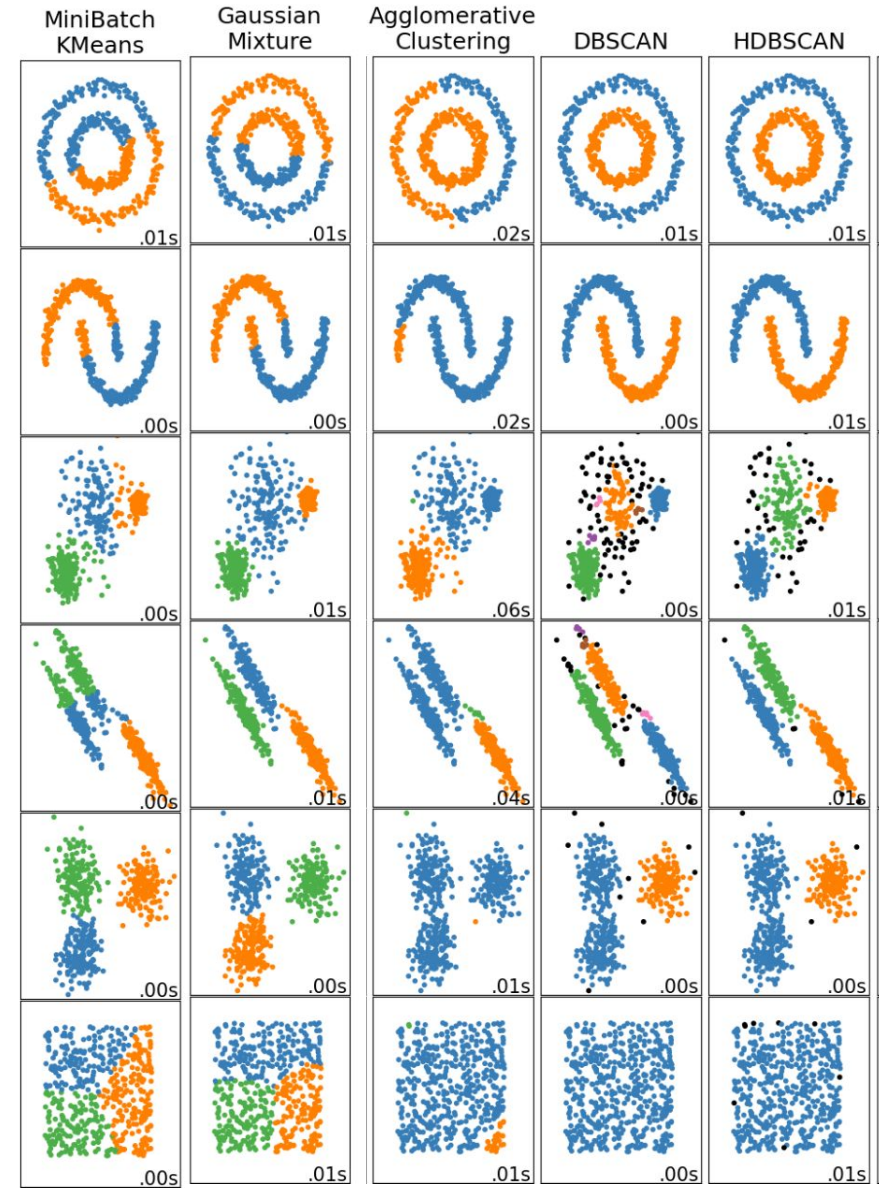
Summary

- **Clustering** is the process of **grouping data** together based on some notion of **similarity**. There are many approaches to clustering, including hierarchical, hard, soft, probabilistic, density-based, and more.
- There are many **clustering algorithms** available including: k-means, GMMs, agglomerative clustering, DBSCAN, and HDBSCAN. They each have their own **strengths and drawbacks**.
- **Evaluating clusters** can be tricky especially in **unsupervised learning**. The Calinski-Harabasz index and Davies-Bouldin index are 2 metrics that can be used to evaluate cluster definition and separation.

Comparison

These plots are taken from an [sklearn demo](#) and show different clustering algorithms on interesting toy datasets.

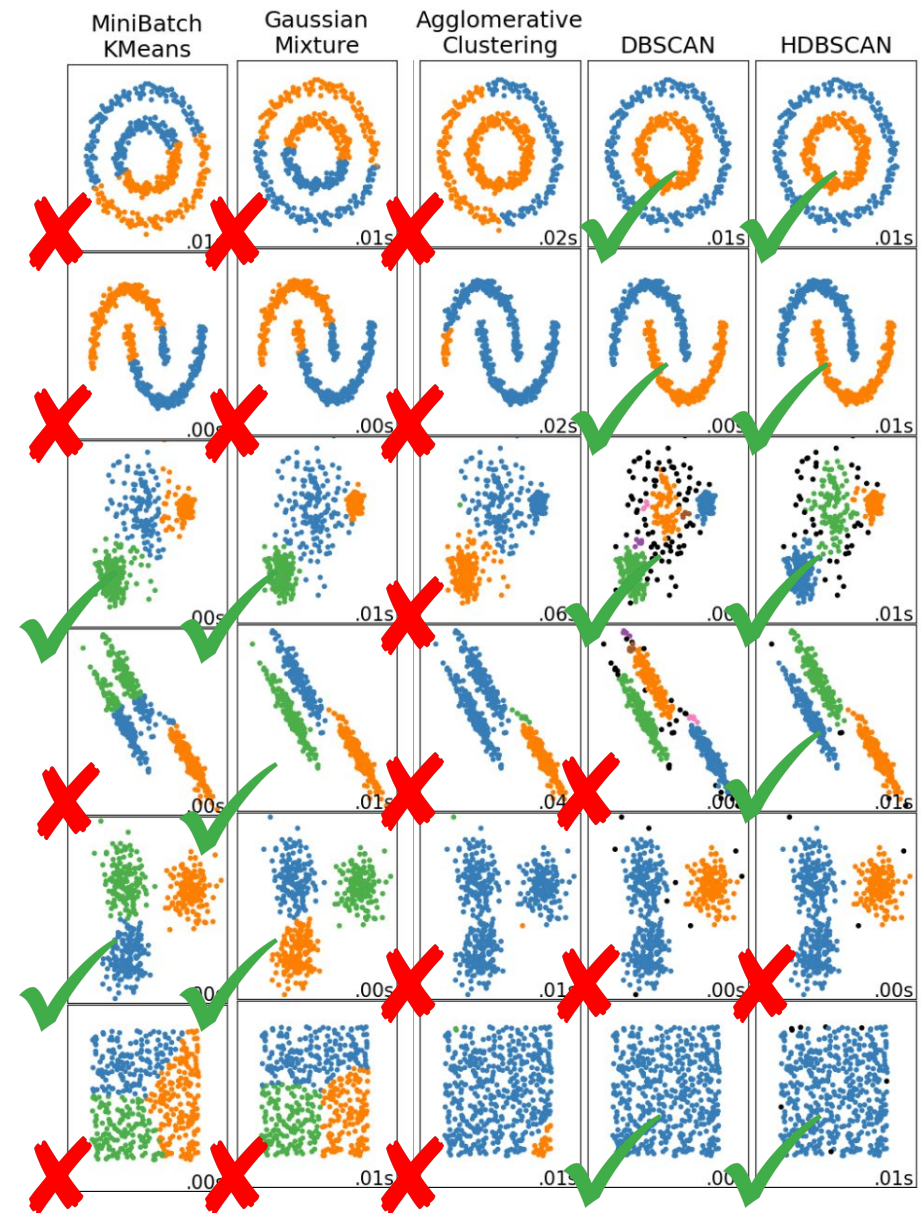
We can use these to compare differences in how the different algorithms perform.



Comparison

Just for fun, here are the "scores" of each algorithm on the datasets. Note that this isn't reflective of their performance as a whole.

K-Means	GMM	Agglomerative	DBSCAN	HDBSCAN
2/6	3/6	0/6	4/6	5/6



Further Resources

[Google ML Overview - K-means](#)

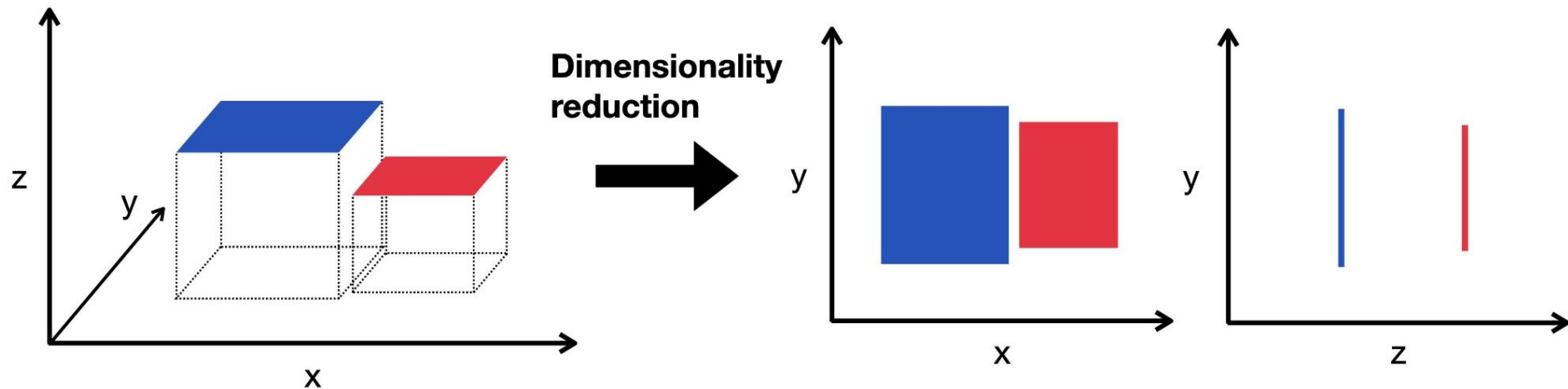
[Comparison of Different Clustering Algorithms using Sklearn](#)

Dimensionality Reduction

What is Dimensionality Reduction?

Dimensionality reduction is the process of representing data using a lower number of dimensions while attempting to capture the original data's properties.

This can involve selecting important features (feature **selection**) or constructing new ones (feature **extraction**).



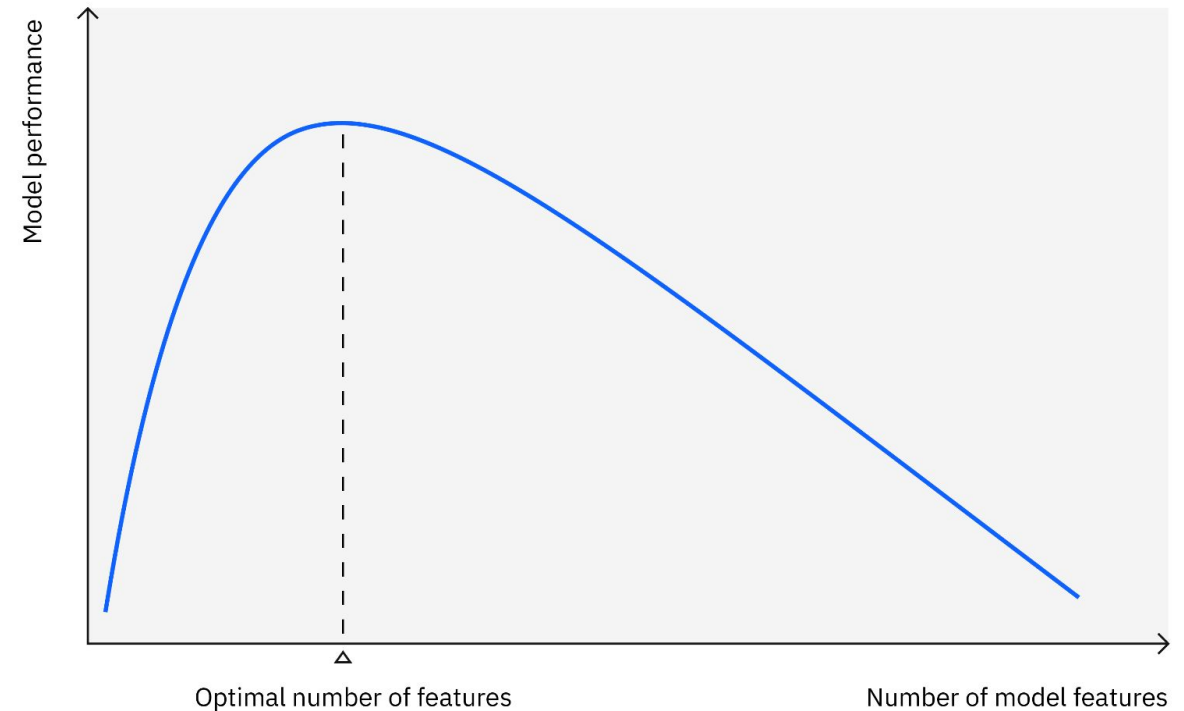
Why do we Need Dimensionality Reduction?

High-dimensional data can cause several issues in machine learning:

- Increased computation time and memory usage
- Risk of overfitting (poor generalization)
- Reduced model interpretability
- The **curse of dimensionality**: as dimensionality grows, data becomes sparse, and distances or similarities become less meaningful.

Curse of Dimensionality

- As the number of **dimensions increases**, the data becomes **more sparse**.
- As a result, distance measures (which are fundamental to many ML algorithms) become **less meaningful**.



Why do we Need Dimensionality Reduction?

We want to **reduce the number of features** without **losing information**.

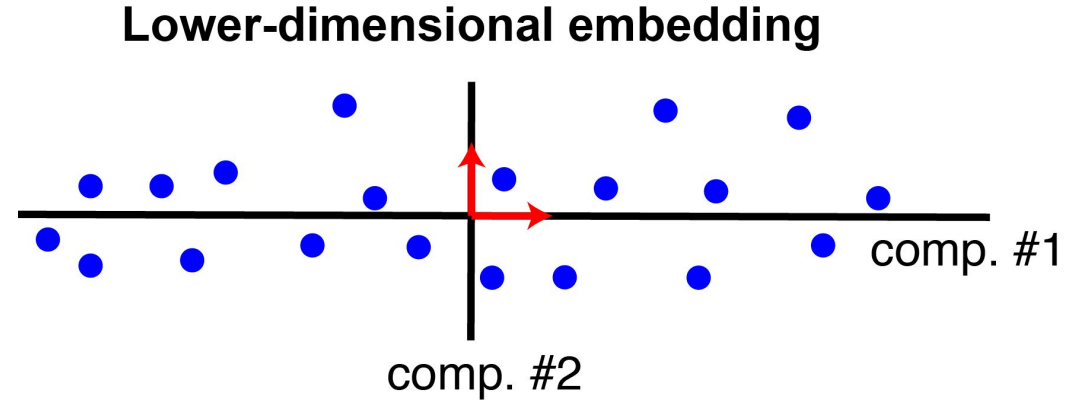
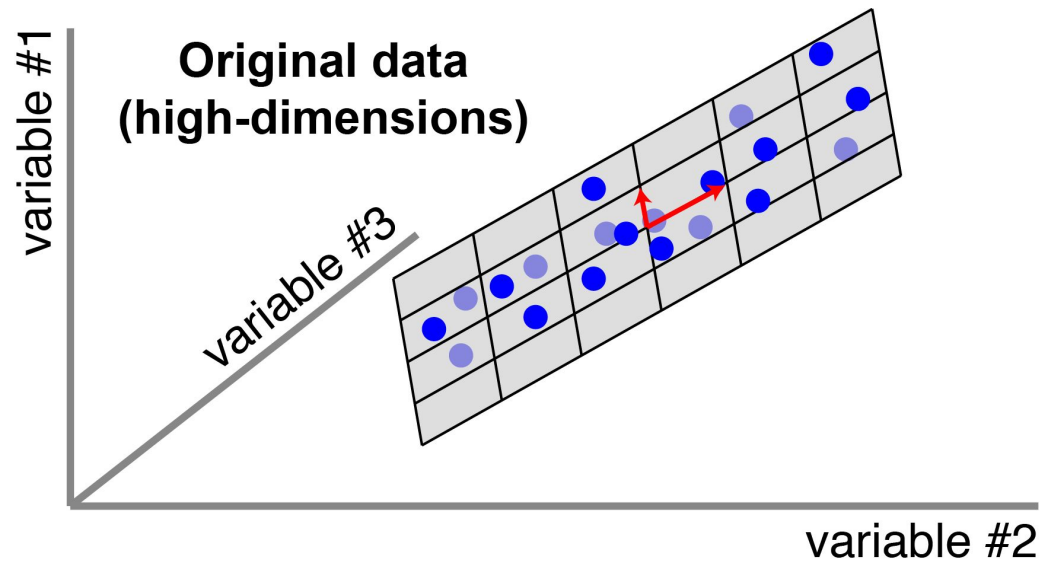
- We need to get rid of redundancies in the data
- We can also detect important features by taking account the correlations between them

Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a **linear** dimensionality reduction method that combines and transforms a dataset's original features to produce new features, called **principal components (PCs)**. PCA then projects data onto a new space defined by the PCs.

- The PCs are **linear combinations** of the original features.
- The first PC (component) has the highest variance (explains the most), and decrease as you move from first to last.
- It's very important to **standardize data before applying PCA** (otherwise, variables with large ranges dominate the PCs).

Principal Component Analysis (PCA)



Principal Component Analysis (PCA)

Each **principal component (PC)** explains some amount of variance in the data. Typically, we choose a threshold (e.g. 95%) and select the number of PCs which explain features up to that threshold.

Principal Component Analysis (PCA)

For example, suppose we perform PCA on a dataset and get the resulting explained variance:

	0	1	2	3	4	5	6	7	8	9	10
Explained Variance	0.4427	0.1897	0.0939	0.0660	0.0550	0.0402	0.0225	0.0159	0.0139	0.0117	0.0098
	11	12	13	14	15	16	17	18	19	20	
	0.0087	0.0080	0.0052	0.0031	0.0027	0.0020	0.0018	0.0016	0.0010	0.0010	

If we wanted to retain 95% of the variance of the original data, we'd keep the first 9 components since the sum of their explained variance $\geq 95\%$.

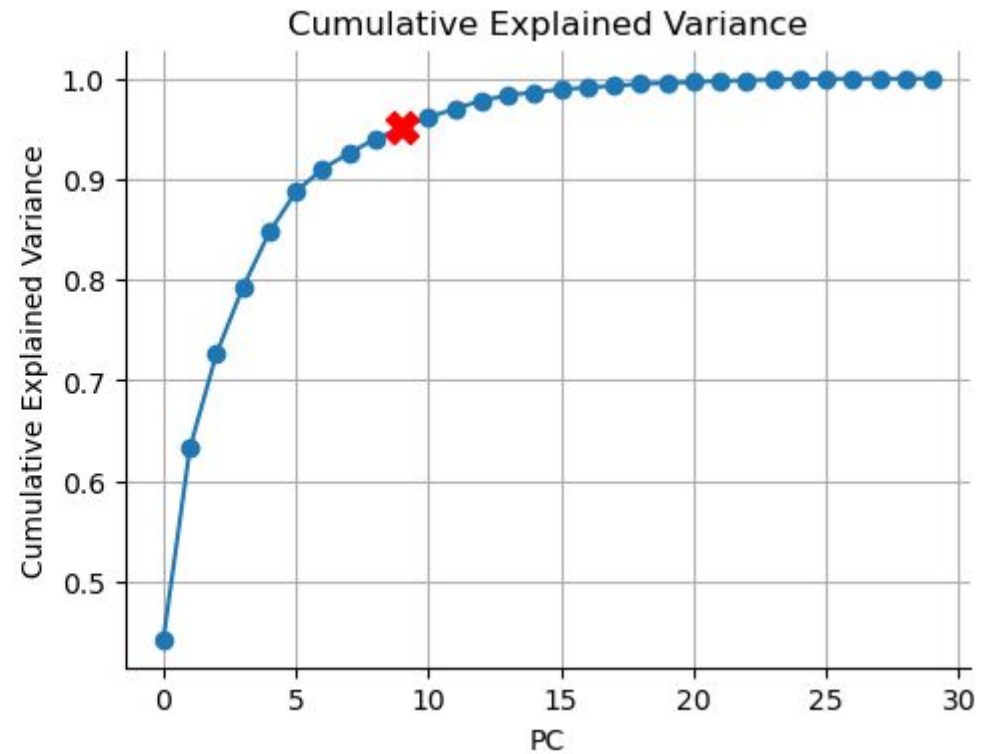
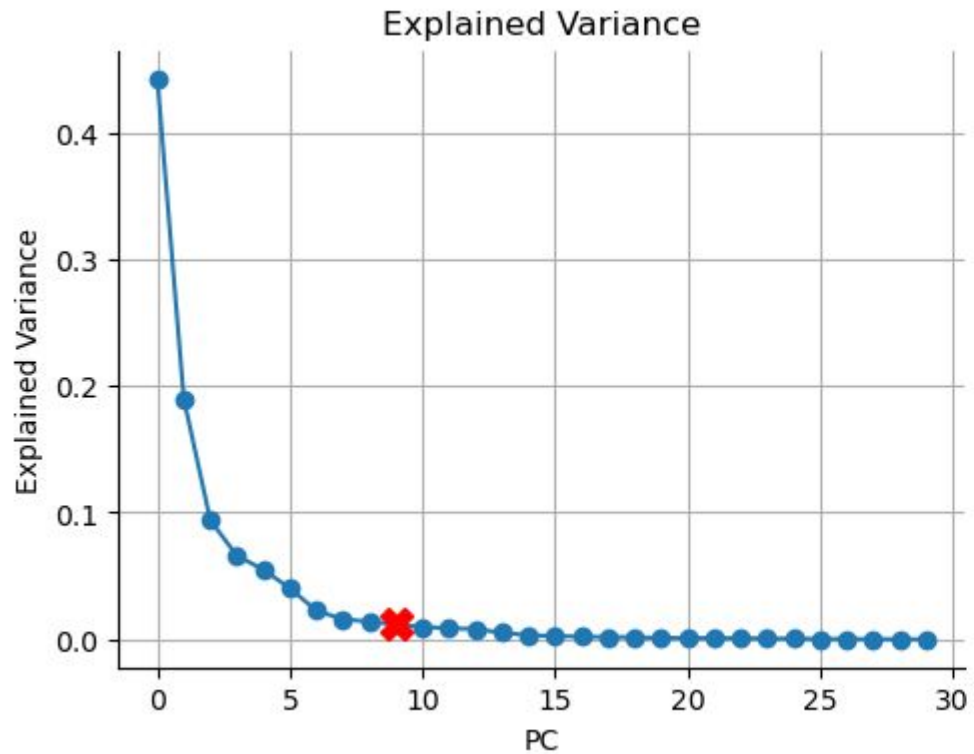
Principal Component Analysis (PCA)

Often, we use the **cumulative explained variance** to make it easier.

	0	1	2	3	4	5	6	7	8	9	10
Explained Variance	0.4427	0.1897	0.0939	0.0660	0.0550	0.0402	0.0225	0.0159	0.0139	0.0117	0.0098
Cumulative Explained Variance	0.4427	0.6324	0.7264	0.7924	0.8473	0.8876	0.9101	0.9260	0.9399	0.9516	0.9614

Principal Component Analysis (PCA)

We can plot the number of PCs kept vs. amount of variance explained.



PCA

Pros:

- Removes redundancy by transforming correlated features into uncorrelated components.

Cons:

- After performing PCA, the resulting principal components (PCs) lose their meaning.
- PCA only captures linear relationships and is not suitable for finding nonlinear structure in data.

t-SNE

T-distributed stochastic neighbor embedding (t-SNE) is a form of non-linear dimensionality reduction. t-SNE is based on defining a probability distribution around **each point**.

t-SNE

One of t-SNE's main parameters is perplexity. **Perplexity** controls the balance between **local vs. more global structure**.

- The best value for perplexity can be decided visually, there is **no algorithm to decide the best value**.
- You cannot see relative sizes of clusters in a t-SNE plot → cluster size does not matter
- Distances between clusters might not mean anything

t-SNE: Perplexity Matters!!



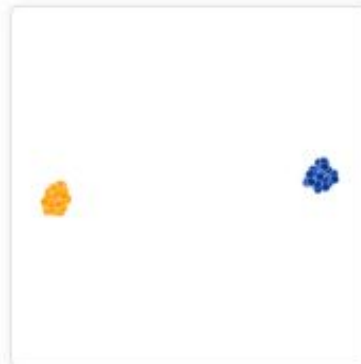
Original



Perplexity: 2
Step: 5,000



Perplexity: 5
Step: 5,000



Perplexity: 30
Step: 5,000



Perplexity: 50
Step: 5,000



Perplexity: 100
Step: 5,000

t-SNE

Pros:

- Good for visualization since it reduces data to 2 or 3 dimensions.
- Handles complex, nonlinear patterns.

Cons:

- Poor global structure: Distances between clusters and cluster sizes are not meaningful.
- Sensitive to hyperparameters (especially perplexity).
- Computationally expensive on large datasets (slower than PCA or UMAP).
 - Can use PCA first and then apply t-SNE to speed it up.

UMAP

Uniform Manifold Approximation and Projection (UMAP) is a non-linear dimensionality reduction method. UMAP constructs a high dimensional graph representation of the data then optimizes a low-dimensional graph to be as structurally similar as possible.

Like the other methods discussed so far, we will not dive deeper into *how* UMAP works as it is very math-heavy and beyond the scope of this course.

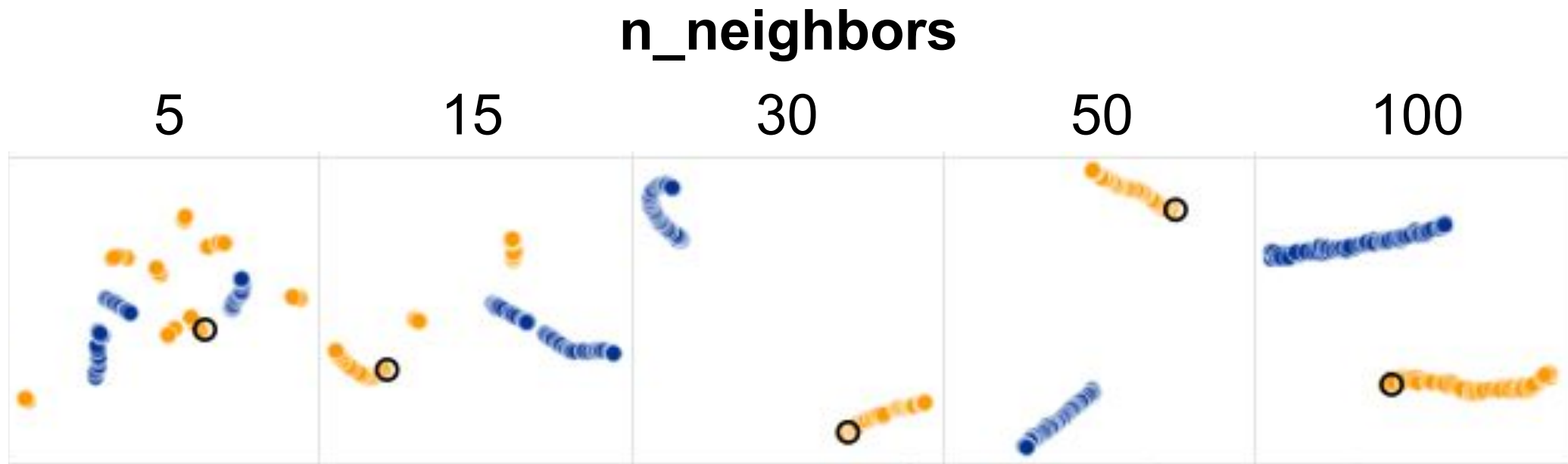
UMAP

UMAP uses 2 main parameters:

1. **n_neighbors** - the number of approximate nearest neighbors used to construct the initial high-dimensional graph. Controls how UMAP balances local versus global structure.
 - Low values → focus more on local structure by constraining the number of neighboring points considered when analyzing the data in high dimensions
 - High values → keeping big-picture structure and lose fine detail.
2. **min_dist** - the minimum distance between points in low-dimensional space. Controls how tightly points are clumped together, with low values leading to more tightly packed embeddings.

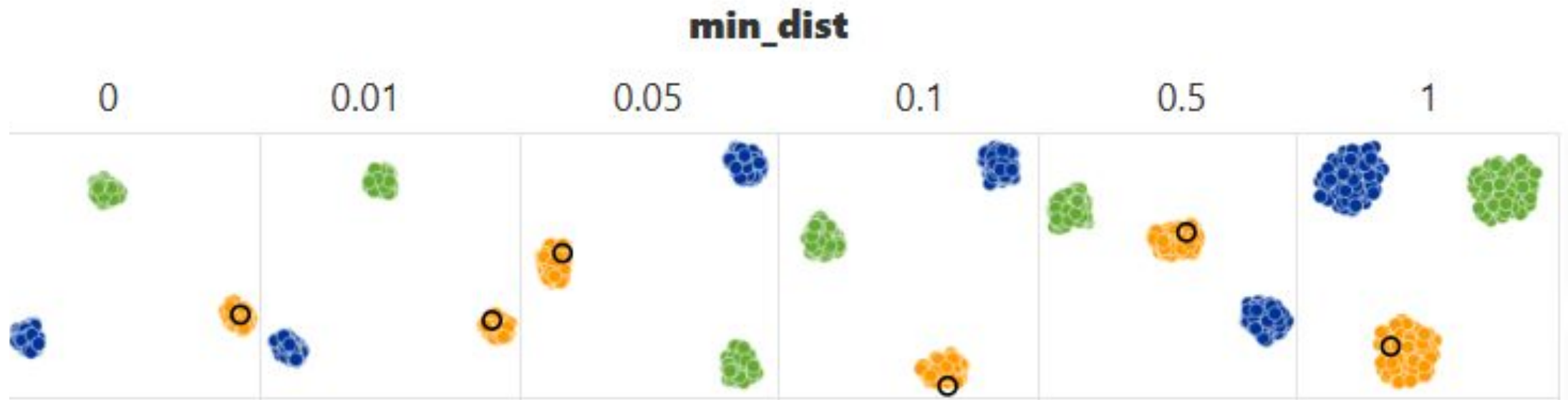
UMAP: `n_neighbors`

In the plots below, we can see that lower values of `n_neighbors` tend to focus more on the **local structure**, whereas higher values focus more on **global structure**.



UMAP: min_dist

We can see in the plots below, that lower values of **min_dist** produce more tightly packed clusters, whereas higher values produce more spread out cluster.



UMAP

Pros:

- Better at preserving global structure than t-SNE.
- Scales well.
- Much faster than t-SNE.
- Parameters are easier to understand the t-SNE's perplexity.

Cons:

- Cluster sizes in a UMAP plot mean nothing.
- Sensitive to hyperparameter values.

Autoencoders

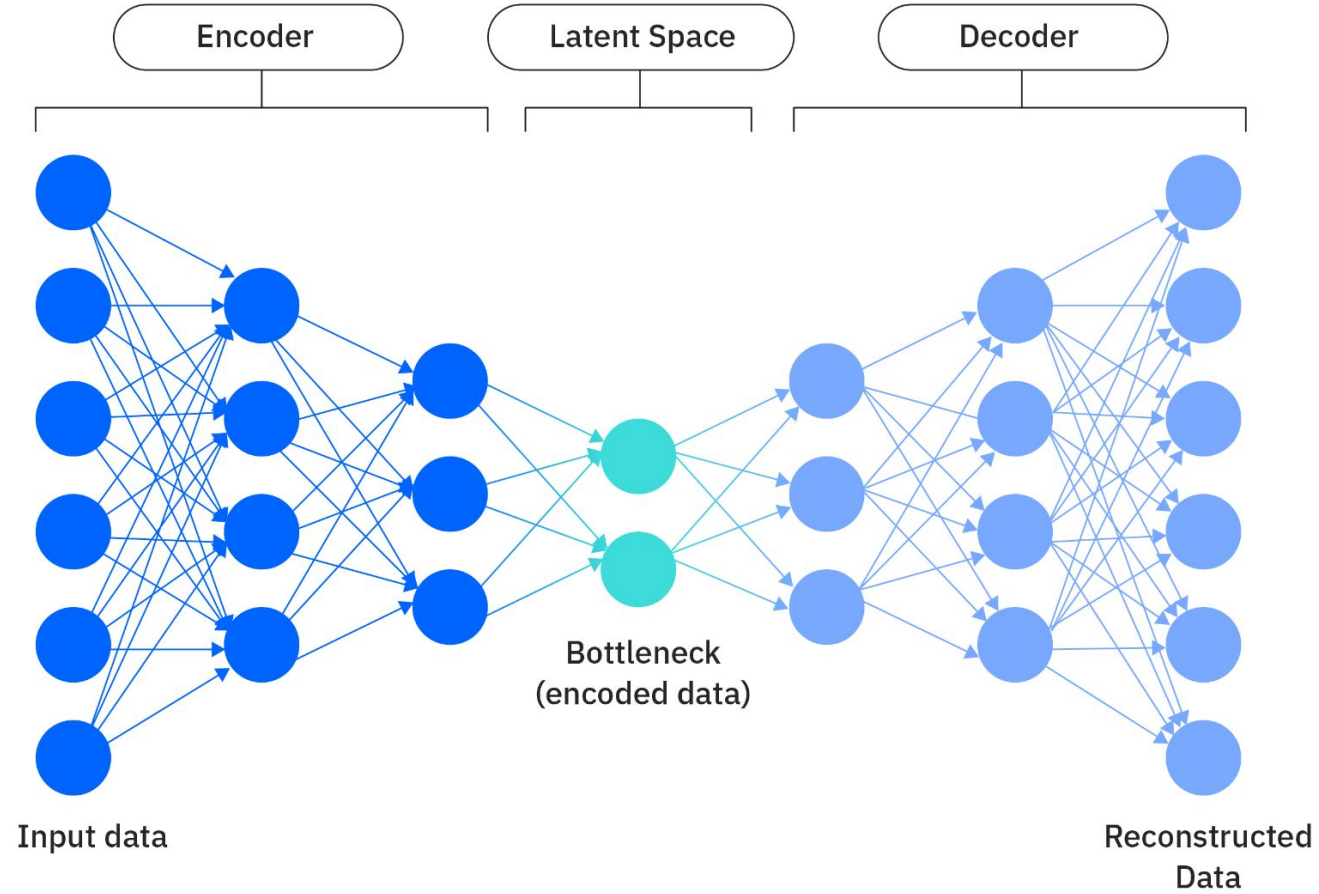
Autoencoders are a type of **neural network architecture** that are trained with unsupervised learning to **reconstruct their own input data**.

They have 2 parts:

1. **Encoder** - efficiently compresses (**encodes**) input data down to its essential features
2. **Decoder** - reconstructs (**decodes**) the original input from the compressed representation.

The **encoder part** of autoencoders can be used for dimensionality reduction.

Autoencoders



Autoencoders

Pros:

- Can capture complex non-linear patterns.
- **Learns** task-specific representations (adapts to each dataset) instead of applying a fixed transformation.

Cons:

- Can be difficult to tune.
- Requires more training data than other dimensionality reduction methods.
- Prone to overfitting.

Summary

- **Dimensionality reduction** refers to reducing the number of dimensions in a dataset. It is often used as a **preprocessing step** so that other ML algorithms can perform better (avoids the **curse of dimensionality**). It can be **linear** (e.g. PCA) or **non-linear** (e.g. t-SNE, UMAP).
- **Scaling** before applying dimensionality reduction is very important, as features with different scales can distort the outcomes.
- **PCA** is one of the most commonly used dimensionality reduction algorithms which produces new features (PCs) by linearly combining the original features.

Further Resources

[Interactive Tool for t-SNE + Explanation of its Pitfalls](#)

[IBM - What is Dimensionality Reduction?](#)

[Understanding UMAP](#)